Exp No.: 5 Integration with External Systems using Hyperledger Fabric

Aim

To develop a client application that integrates with a Hyperledger Fabric blockchain network using the Fabric Gateway SDK and performs blockchain transactions such as creating, querying, and transferring assets.

Software Requirements

- Kali Linux / Ubuntu
- Docker
- Docker Compose
- Node.js (Version 18 or later)
- npm
- Git
- Hyperledger Fabric Samples
- Hyperledger Fabric Binaries
- Fabric Gateway SDK (Node.js)
- Visual Studio Code

Hardware Requirements

- Processor: Intel Core i3 or above
- RAM: Minimum 8 GB
- Storage: Minimum 20 GB Free Disk Space
- Internet Connection

Theory

Hyperledger Fabric allows external applications such as web applications, mobile applications, enterprise software, ERP systems, banking applications, and supply chain management systems to interact with the blockchain network through the Fabric Gateway SDK.

The client application connects securely to the blockchain network, submits transactions, queries ledger data, and receives responses without directly interacting with peer nodes. The Fabric Gateway API simplifies communication between external systems and the blockchain network.

Procedure

Step 1: Navigate to the Fabric Test Network

```
cd ~/fabric-dev/fabric-samples/test-network
```

Step 2: Start the Fabric Network

```
./network.sh up createChannel –ca
```

Expected Output

```
Creating channel 'mychannel'

Starting peer0.org1

Starting peer0.org2
```

Starting orderer.example.com

Network Started Successfully
Step 3: Deploy the JavaScript Chaincode
./network.sh deployCC \
-ccl javascript \
-ccn basic \
-ccp ../asset-transfer-basic/chaincode-javascript

Expected Output
Packaging Chaincode

Installing Chaincode

Approving Chaincode

Committing Chaincode

Chaincode committed successfully

Step 4: Navigate to the Application Directory
cd ../asset-transfer-basic/application-gateway-javascript

Step 5: Install Node.js Packages
Install the required dependencies.
npm install

Expected Output
added 150 packages

found 0 vulnerabilities

Step 6: Run the Client Application
Execute the JavaScript application.
node app.js

Expected Output
--> Submit Transaction: InitLedger

*** Ledger initialized successfully

Step 7: Query All Assets
The application automatically queries all assets stored in the blockchain.

Expected Output

[
{
"ID":"asset1",
"Owner":"Tomoko",

"Color":"Blue",
"Size":5,
"AppraisedValue":300
},
{
"ID":"asset2",
"Owner":"Brad",
"Color":"Red",
"Size":5,
"AppraisedValue":400
}
]

Step 8: Create a New Asset
The client application submits a transaction to create a new asset.
Expected Output
Submit Transaction: CreateAsset

Asset Created Successfully

Step 9: Transfer Asset Ownership
The application transfers ownership of an existing asset.
Expected Output
Submit Transaction: TransferAsset

Owner Updated Successfully

Step 10: Query Updated Asset
Retrieve the updated asset information.

Expected Output

{
"ID":"asset7",
"Owner":"David",
"Color":"Black",
"Size":15,
"AppraisedValue":900
}

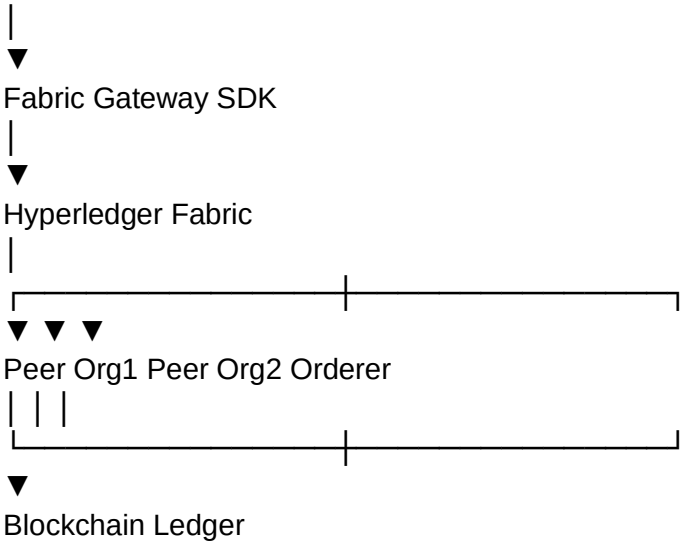
Step 11: Stop the Fabric Network
cd ../../test-network

./network.sh down

Sample JavaScript Client Program

```
const { Gateway, Wallets } = require('fabric-network');

async function main() {

console.log("Connecting to Hyperledger Fabric Network...");

console.log("Submitting Transaction");

console.log("Transaction Successful");

console.log("Querying Ledger");

console.log("Asset Retrieved Successfully");

}

main();
```

Flow Diagram



Application Workflow

▼
Deploy Chaincode
│
▼
Install Node.js Packages
│
▼
Run Client Application

│
▼
Connect using Gateway SDK
│
▼
Submit Transaction
│
▼
Query Ledger
│
▼
Receive Response
│
▼
Display Asset Details
│
▼
Stop Fabric Network
Sample Output
Fabric Network Started Successfully

Gateway Connected Successfully

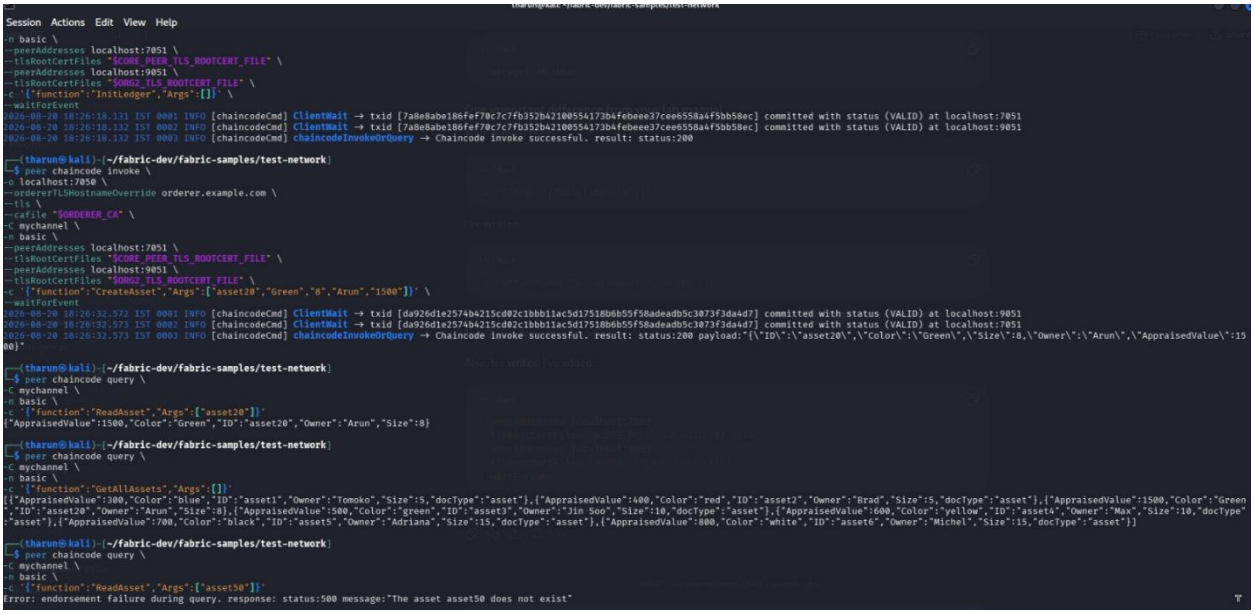
Ledger Initialized

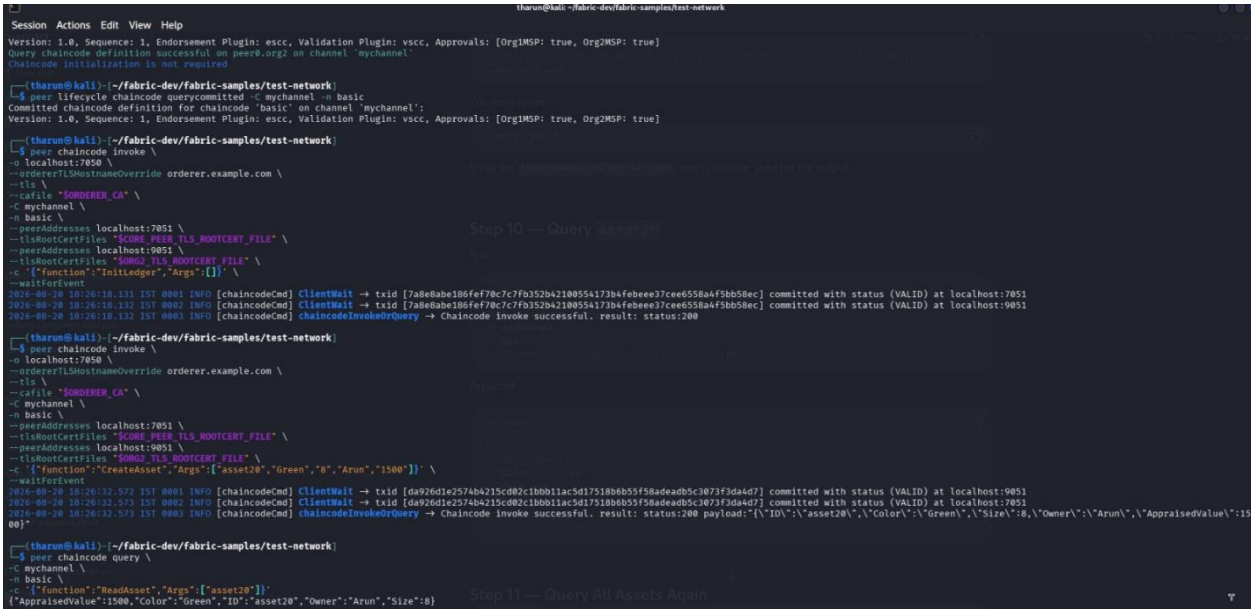
Assets Retrieved Successfully

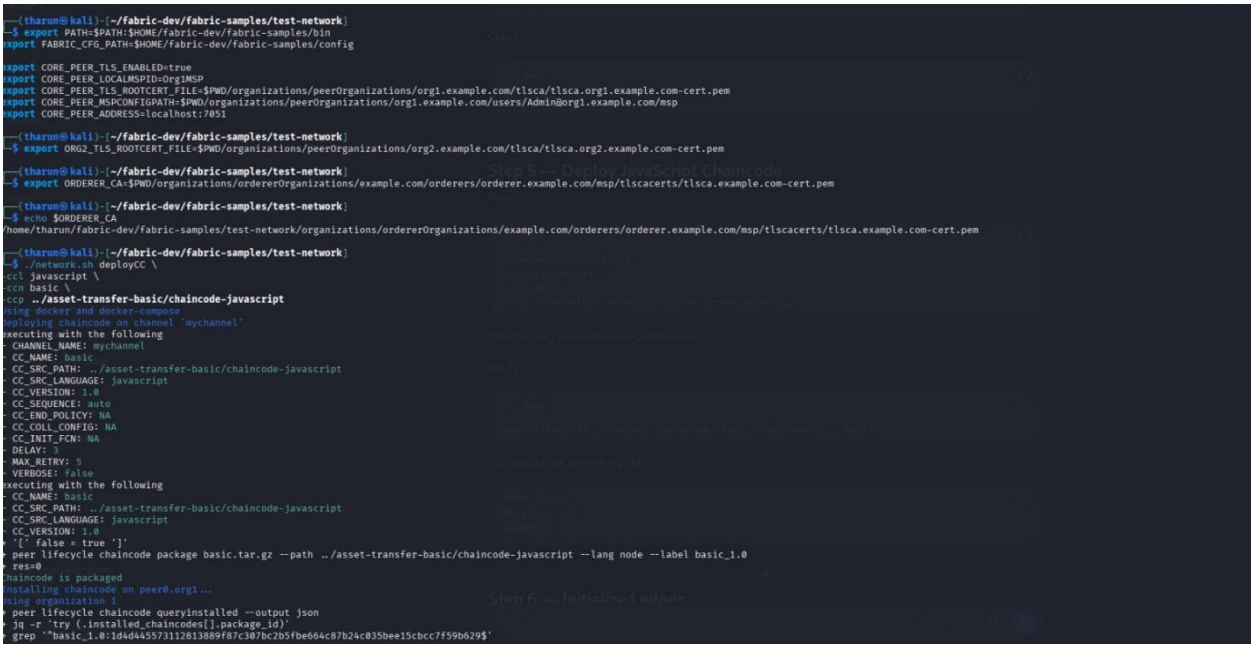
New Asset Created Successfully

Asset Ownership Updated Successfully

Application Executed Successfully


Session Actions Edit View Help
-n basic \
--peerAddresses localhost:7051 \
--tlsRootCertFiles "$CORE_PEER_TLS_ROOTCERT_FILE" \
--peerAddresses localhost:9051 \
--tlsRootCertFiles "$ORG2_TLS_ROOTCERT_FILE" \
-c '{"function":"InitLedger","Args":[]}' \
--waitForEvent
2026-08-20 18:26:18.131 IST 0001 INFO [chaincodeCmd] ClientWait → txid [7a8e8abe186fef70c7c7fb352b42100554173b4febeee37cee6558a4f5bb58ec] committed with status (VALID) at localhost:7051
2026-08-20 18:26:18.132 IST 0002 INFO [chaincodeCmd] ClientWait → txid [7a8e8abe186fef70c7c7fb352b42100554173b4febeee37cee6558a4f5bb58ec] committed with status (VALID) at localhost:9051
2026-08-20 18:26:18.132 IST 0003 INFO [chaincodeCmd] chaincodeInvokeOrQuery → Chaincode invoke successful. result: status:200
┌──(tharun㉿kali)-[~/fabric-dev/fabric-samples/test-network]
└─$ peer chaincode invoke \
-o localhost:7050 \
--ordererTLSHostnameOverride orderer.example.com \
--tls \
--cafile "$ORDERER_CA" \
-C mychannel \
-n basic \
--peerAddresses localhost:7051 \
--tlsRootCertFiles "$CORE_PEER_TLS_ROOTCERT_FILE" \
--peerAddresses localhost:9051 \
--tlsRootCertFiles "$ORG2_TLS_ROOTCERT_FILE" \
-c '{"function":"CreateAsset","Args":["asset20","Green","8","Arun","1500"]}' \
--waitForEvent
2026-08-20 18:26:32.572 IST 0001 INFO [chaincodeCmd] ClientWait → txid [da926d1e2574b4215cd02c1bbb11ac5d17518b6b55f58adeadb5c3073f3da4d7] committed with status (VALID) at localhost:9051
2026-08-20 18:26:32.573 IST 0002 INFO [chaincodeCmd] ClientWait → txid [da926d1e2574b4215cd02c1bbb11ac5d17518b6b55f58adeadb5c3073f3da4d7] committed with status (VALID) at localhost:7051
2026-08-20 18:26:32.573 IST 0003 INFO [chaincodeCmd] chaincodeInvokeOrQuery → Chaincode invoke successful. result: status:200 payload:"{\"ID\":\"asset20\",\"Color\":\"Green\",\"Size\":8,\"Owner\":\"Arun\",\"AppraisedValue\":15
00}"
┌──(tharun㉿kali)-[~/fabric-dev/fabric-samples/test-network]
└─$ peer chaincode query \
-C mychannel \
-n basic \
-c '{"function":"ReadAsset","Args":["asset20"]}'
{"AppraisedValue":1500,"Color":"Green","ID":"asset20","Owner":"Arun","Size":8}
┌──(tharun㉿kali)-[~/fabric-dev/fabric-samples/test-network]
└─$ peer chaincode query \
-C mychannel \
-n basic \
-c '{"function":"GetAllAssets","Args":[]}'
[{"AppraisedValue":300,"Color":"blue","ID":"asset1","Owner":"Tomoko","Size":5,"docType":"asset"},{"AppraisedValue":400,"Color":"red","ID":"asset2","Owner":"Brad","Size":5,"docType":"asset"},{"AppraisedValue":1500,"Color":"Green
","ID":"asset20","Owner":"Arun","Size":8},{"AppraisedValue":500,"Color":"green","ID":"asset3","Owner":"Jin Soo","Size":10,"docType":"asset"},{"AppraisedValue":600,"Color":"yellow","ID":"asset4","Owner":"Max","Size":10,"docType"
:"asset"},{"AppraisedValue":700,"Color":"black","ID":"asset5","Owner":"Adriana","Size":15,"docType":"asset"},{"AppraisedValue":800,"Color":"white","ID":"asset6","Owner":"Michel","Size":15,"docType":"asset"}]
┌──(tharun㉿kali)-[~/fabric-dev/fabric-samples/test-network]
└─$ peer chaincode query \
-C mychannel \
-n basic \
-c '{"function":"ReadAsset","Args":["asset50"]}'
Error: endorsement failure during query. response: status:500 message:"The asset asset50 does not exist"


Session Actions Edit View Help
Version: 1.0, Sequence: 1, Endorsement Plugin: escc, Validation Plugin: vscc, Approvals: [Org1MSP: true, Org2MSP: true]
Query chaincode definition successful on peer0.org2 on channel 'mychannel'
Chaincode initialization is not required
┌──(tharun㉿kali)-[~/fabric-dev/fabric-samples/test-network]
└─$ peer lifecycle chaincode querycommitted -C mychannel -n basic
Committed chaincode definition for chaincode 'basic' on channel 'mychannel':
Version: 1.0, Sequence: 1, Endorsement Plugin: escc, Validation Plugin: vscc, Approvals: [Org1MSP: true, Org2MSP: true]
┌──(tharun㉿kali)-[~/fabric-dev/fabric-samples/test-network]
└─$ peer chaincode invoke \
-o localhost:7050 \
--ordererTLSHostnameOverride orderer.example.com \
--tls \
--cafile "$ORDERER_CA" \
-C mychannel \
-n basic \
--peerAddresses localhost:7051 \
--tlsRootCertFiles "$CORE_PEER_TLS_ROOTCERT_FILE" \
--peerAddresses localhost:9051 \
--tlsRootCertFiles "$ORG2_TLS_ROOTCERT_FILE" \
-c '{"function":"InitLedger","Args":[]}' \
--waitForEvent
2026-08-20 18:26:18.133 IST 0001 INFO [chaincodeCmd] ClientWait → txid [7a8e8abe186fef70c7c7fb352b42100554173b4febeee37cee6558a4f5bb58ec] committed with status (VALID) at localhost:7051
2026-08-20 18:26:18.132 IST 0002 INFO [chaincodeCmd] ClientWait → txid [7a8e8abe186fef70c7c7fb352b42100554173b4febeee37cee6558a4f5bb58ec] committed with status (VALID) at localhost:9051
2026-08-20 18:26:18.132 IST 0003 INFO [chaincodeCmd] chaincodeInvokeOrQuery → Chaincode invoke successful. result: status:200
┌──(tharun㉿kali)-[~/fabric-dev/fabric-samples/test-network]
└─$ peer chaincode invoke \
-o localhost:7050 \
--ordererTLSHostnameOverride orderer.example.com \
--tls \
--cafile "$ORDERER_CA" \
-C mychannel \
-n basic \
--peerAddresses localhost:7051 \
--tlsRootCertFiles "$CORE_PEER_TLS_ROOTCERT_FILE" \
--peerAddresses localhost:9051 \
--tlsRootCertFiles "$ORG2_TLS_ROOTCERT_FILE" \
-c '{"function":"CreateAsset","Args":["asset20","Green","8","Arun","1500"]}' \
--waitForEvent
2026-08-20 18:26:32.572 IST 0001 INFO [chaincodeCmd] ClientWait → txid [da926d1e2574b4215cd02c1bbb11ac5d17518b6b55f58adeadb5c3073f3da4d7] committed with status (VALID) at localhost:9051
2026-08-20 18:26:32.573 IST 0002 INFO [chaincodeCmd] ClientWait → txid [da926d1e2574b4215cd02c1bbb11ac5d17518b6b55f58adeadb5c3073f3da4d7] committed with status (VALID) at localhost:7051
2026-08-20 18:26:32.573 IST 0003 INFO [chaincodeCmd] chaincodeInvokeOrQuery → Chaincode invoke successful. result: status:200 payload:"{\"ID\":\"asset20\",\"Color\":\"Green\",\"Size\":8,\"Owner\":\"Arun\",\"AppraisedValue\":15
00}"
┌──(tharun㉿kali)-[~/fabric-dev/fabric-samples/test-network]
└─$ peer chaincode query \
-C mychannel \
-n basic \
-c '{"function":"ReadAsset","Args":["asset20"]}'
{"AppraisedValue":1500,"Color":"Green","ID":"asset20","Owner":"Arun","Size":8}



Result

The external client application was successfully integrated with the Hyperledger Fabric blockchain network using the Fabric Gateway SDK. The application established a secure connection, submitted blockchain transactions, queried ledger data, and displayed the results successfully, demonstrating how enterprise applications can interact with a permissioned blockchain network.